Loss Functions in Deep Learning

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. Why the loss function matters . . . 3
   - 3.1 The loss in machine learning . . . 3
   - 3.2 The loss inside a neural network . . . 4
4. Loss function versus cost function . . . 5
5. Mean squared error . . . 6
   - 5.1 The formula . . . 6
   - 5.2 Far observations push harder . . . 6
   - 5.3 Advantages and disadvantages . . . 8
   - 5.4 MSE in a Keras network . . . 8
6. Mean absolute error . . . 9
7. Huber loss . . . 10
   - 7.1 When neither MSE nor MAE fits . . . 10
   - 7.2 The formula . . . 10
   - 7.3 The three losses on data with outliers . . . 11
8. Binary cross-entropy . . . 12
   - 8.1 Why not the squared error? . . . 14
   - 8.2 Advantages and disadvantages . . . 15
9. Categorical cross-entropy . . . 15
   - 9.1 The setup . . . 15
   - 9.2 Two students . . . 17
10. Sparse categorical cross-entropy . . . 18
11. Choosing a loss . . . 19
12. Summary . . . 21
13. Sources . . . 21
14. Key terms . . . 22

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Categorical and sparse categorical cross-entropy; Log loss (binary cross entropy); Forward propagation.
- **Leads to:** Backpropagation.
- **Compare with:** Regression metrics.

1. Overview

Key point: A loss function scores how wrong the network is on one **observation** (one record, one row of the data table). Training changes the weights and biases until that score is as small as possible. Which loss we use depends on the problem, and it fixes the activation of the output layer (the function applied to that layer's sums, activation functions).

A loss function measures how wrong a model's predictions are. The loss is a function of the model's parameters: change any weight or bias, and the loss changes (see the loss as a function of the parameters). A large loss means the model is doing badly; a small one means it is doing well.

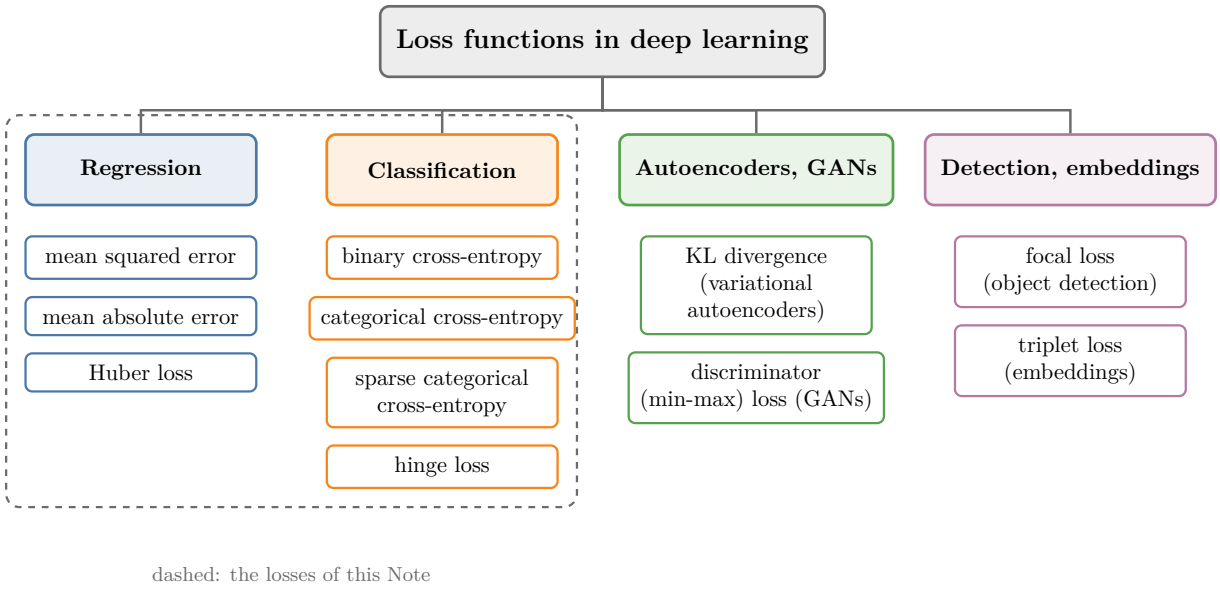


Figure 1: The main loss functions of deep learning, grouped by the problem they solve

Figure 1 groups the loss functions by problem. This Note covers the dashed part: three losses for regression and three for classification, with the output layer and Keras setting each one needs.

2. Prerequisites

- **Forward propagation** (G-797): how a network turns an observation into a prediction.
- MSE and MAE as metrics: **mean squared error** (G-1201) and **mean absolute error** (G-1194).
- **Binary cross-entropy** (G-303): log loss. **Categorical cross-entropy** (G-349) and softmax: the softmax function.

3. Why the loss function matters

> **Key point:** We cannot improve what we cannot measure. The loss is the number training watches: every weight update is decided by it.

3.1 The loss in machine learning

> **Key point:** Start with a random line, measure its loss, move the line, measure again; stop when the loss is smallest.

In simple linear regression, the loss is the **mean squared error** (G-1201), and $\hat{y}_i = mx_i + b$. So the loss is a function of m and b only. Gradient descent starts from a random line, computes its loss, and changes m and b to lower it, again and again (see the update rule of gradient descent).

The loss is the algorithm's eyes. Without it, the algorithm has no way to know which direction is better.

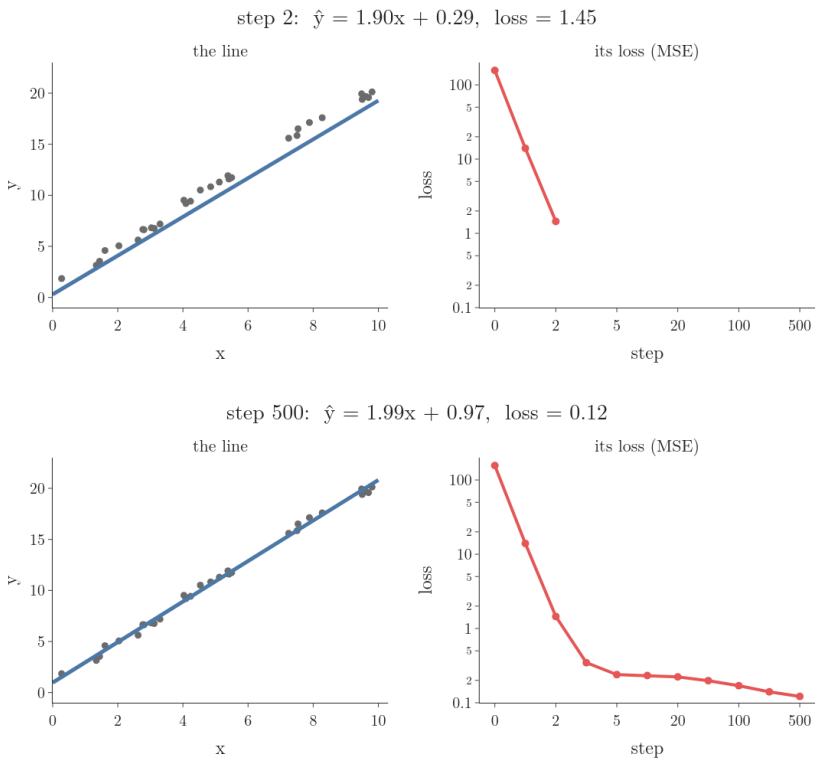


Figure 2: Gradient descent on 30 points that follow y = 2x + 1 (the same 30 points reappear in Figure 6): the line moves (left) while its MSE loss falls (right)

Figure 2 runs this loop from $m = 0$, $b = 0$ with learning rate 0.01. Watch the loss fall from 156 to 1.45 in two steps, then creep down to 0.12 by step 500 as the line settles on the points.

3.2 The loss inside a neural network

> **Key point:** Each observation goes forward through the network, its loss is computed, and the weights and biases are adjusted to lower that loss. Repeat for every observation, for many epochs.

A network is trained in the same spirit. Take the student data (CGPA, IQ, package in LPA, lakhs of rupees per year):

1. Start with random weights and biases.
2. Feed the first student forward (7.1 and 83 into the input nodes) and get a prediction, say $\hat{y} = 3.7$.
3. Put y and $\hat{y}$ into the loss function, here the **mean squared error** (G-1201), because the output is a number.
4. Use that loss to adjust every weight and bias with gradient descent.
5. Take the next student and repeat; go over the whole dataset many times (epochs).

The weights and biases that give the smallest loss are the trained network. How step 4 is done, for every weight at once, is **backpropagation**, taught in the steps of backpropagation.

4. Loss function versus cost function

> **Key point:** The loss is computed on one observation; the cost is the average loss over a batch or the whole training set. Many people use the two words for the same thing.

Strictly, the two words mean different things:

- **Loss function** (G-706, also called **error function**): the error on a single training observation, e.g. $(y_i - \hat{y}_i)^2$.
- **Cost function** (G-492): the average of the losses over a batch of observations, or over the whole training set, e.g. $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$.

Four students show the difference:

Student	y (LPA)	$\hat{y}$	$y - \hat{y}$	Loss $(y - \hat{y})^2$
1	6.3	6.1	0.2	0.04
2	4.1	4.0	0.1	0.01
3	3.5	3.7	−0.2	0.04
4	7.2	7.0	0.2	0.04

1. **In words:** compute the loss of each observation, then average them.
2. **Formula:**
$$J = \frac{1}{n}\sum_{i=1}^{n} L_i$$
3. **Example:**
$$J = \frac{0.04 + 0.01 + 0.04 + 0.04}{4} = 0.0325$$

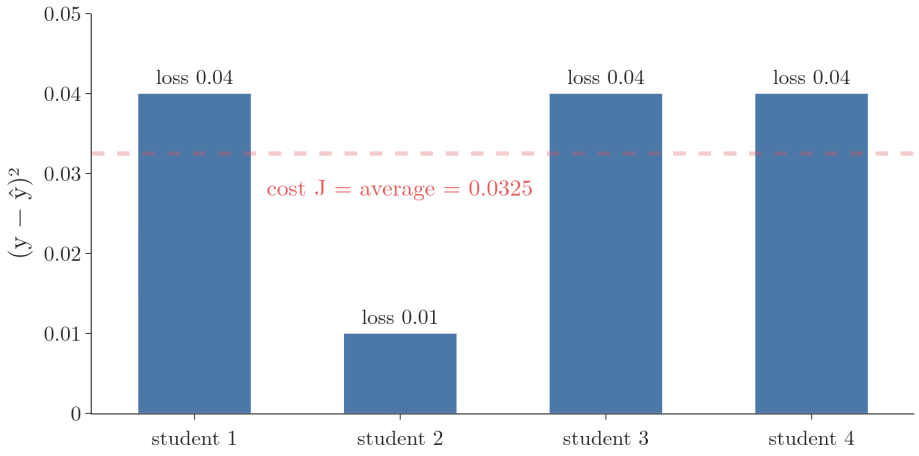


Figure 3: Each bar is one student's loss; the dashed line is the cost, their average

Figure 3 shows the difference at a glance: four losses, one per student, and one cost line through their middle.

Why keep two words? The cost is what training pushes down, and it is an average of per-observation losses. So its gradient (the direction that lowers it) is an average too, and the average over a small random batch of observations is a good estimate of it. This is what lets **mini-batch gradient descent** (G-1222) (one update per small batch) train on very large datasets (Goodfellow, Bengio and Courville, *Deep Learning*, §5.9).

> **Extra:** Other texts, such as the chapter on convex and non-convex cost functions, use "cost function" loosely as another name for the loss. Libraries also blur the line: by default Keras averages the losses over the batch and reports that average as the loss (Keras docs, Losses).

5. Mean squared error

> **Key point:** MSE squares each error: far-off observations get much bigger weight updates. Use it for regression without **outliers** (G-1420), with a linear output node.

5.1 The formula

> **Key point:** Loss $(y - \hat{y})^2$; cost $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$. Also called squared loss or L2 loss.

The **mean squared error** (MSE, G-1201) is defined in the mean squared error. As a training loss it is also called the **squared loss** or **L2 loss**.

The square is there so that errors cannot cancel. In the table of Section 4, the raw errors add up as follows:

$$0.2 + 0.1 = 0.3$$

$$0.3 - 0.2 = 0.1$$

$$0.1 + 0.2 = 0.3$$

Student 3's negative error hides part of the others. Squaring makes every error count as positive.

5.2 Far observations push harder

> **Key point:** The slope of the loss is $-2(y - \hat{y})$. An observation with an error of 42 pushes the weights 42 times harder than an observation with an error of 1.

Squaring has a second effect: errors grow quadratically. An error of 1 costs 1, an error of 2 costs 4, an error of 4 costs 16.

What moves the weights is the slope of the loss, written $\partial L/\partial \hat{y}$ (read it as: how much L changes when $\hat{y}$ alone changes a little; backpropagation multiplies every weight's update by it):

1. **In words:** for MSE, the slope is -2 times the error, so the push grows in proportion to the error.
2. **Formula:**
$$\frac{\partial}{\partial \hat{y}}(y - \hat{y})^2 = -2(y - \hat{y})$$
3. **Example:** a student with package 8 predicted as 7 (error 1):
$$\text{slope} = -2 \times 1 = -2$$
A student with a package of 50 lakhs predicted as 8 (error 42):
$$\text{loss} = 42^2 = 1764$$
$$\text{slope} = -2 \times 42 = -84$$
That is a 42 times bigger push.

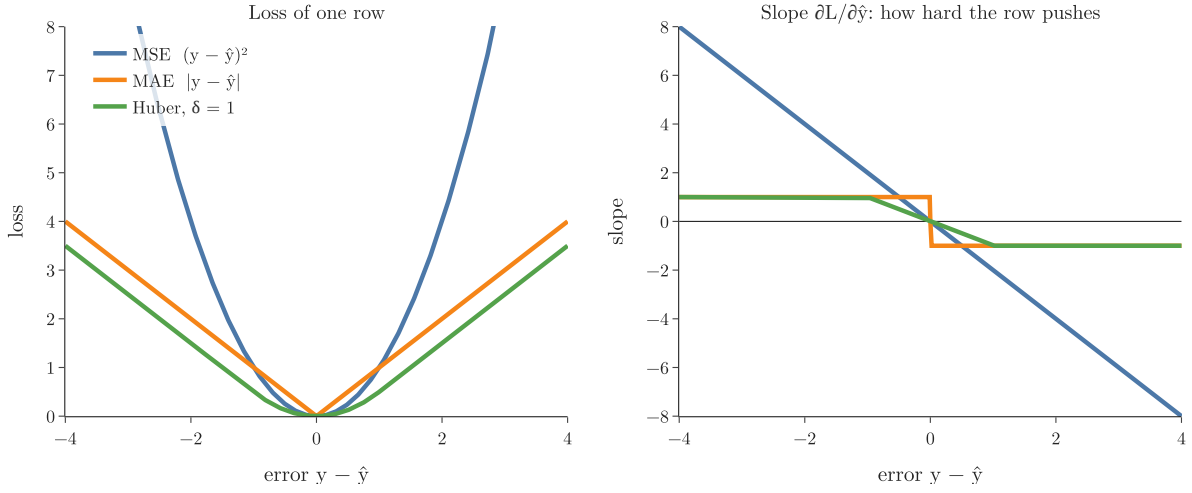


Figure 4: Left: the loss of one observation against its error. Right: its slope, which sets how hard the observation pushes the weights

Figure 4 (blue) shows both: the loss is a parabola, and its slope a straight line through 0. Observations far from the prediction drive the weight updates.

5.3 Advantages and disadvantages

> **Key point:** Easy to read and smooth everywhere, but in squared units and pulled hard by **outliers** (G-1420).

Advantages:

- **Easy to interpret:** the formula says exactly how the error is measured.
- **Differentiable everywhere:** the parabola has no corners, so gradient descent can always compute a slope.
- **One minimum** for a linear model such as linear regression (see a straight line has one bowl).

Disadvantages:

- **Squared units:** a loss of 5 means 5 LPA^2, not 5 LPA. Its square root, the RMSE, is back in LPA.
- **Not robust to outliers:** the 50-lakh student of Section 5.2 dominates the updates, and the network bends towards it, away from the other students (Figure 6 in Section 7).

> **Extra:** "One minimum" holds for a linear model only. As a function of a network's weights, even the MSE has several minima, because the network sits between the loss and the weights (see a tiny neural network has two minima). This is true for every loss in this Note.

5.4 MSE in a Keras network

> **Key point:** The output node must be linear, so it can output any number. Hidden layers can use any activation.

With MSE, the output layer needs a linear activation (no activation at all), because a package can be any number. A sigmoid output would trap every prediction between 0 and 1. The hidden layers can use any activation, such as ReLU or sigmoid.

> **Python:** A regression network with MSE.

```
import keras

model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(3, activation="relu"),
    keras.layers.Dense(1, activation="linear"),  # any number
])
model.compile(optimizer="sgd", loss="mse")
```

`loss="mse"` is short for `"mean_squared_error"`. `activation="linear"` is the default of `Dense`, so it can also be left out.

6. Mean absolute error

> **Key point:** MAE takes the absolute error instead of the square. Its units are those of y and **outliers** (G-1420) pull less, but its slope jumps at 0.

The **mean absolute error** (MAE, G-1194) is defined in the mean absolute error. As a training loss it is also called **L1 loss**: loss $|y_i - \hat{y}_i|$ for one observation, cost $\frac{1}{n}\sum |y_i - \hat{y}_i|$ over n observations.

The only change from MSE is the absolute value in place of the square. The MAE's slope (Figure 4, orange) is -1 or $+1$, whatever the size of the error. The 50-lakh student now pushes exactly as hard as a student whose prediction is off by 1.

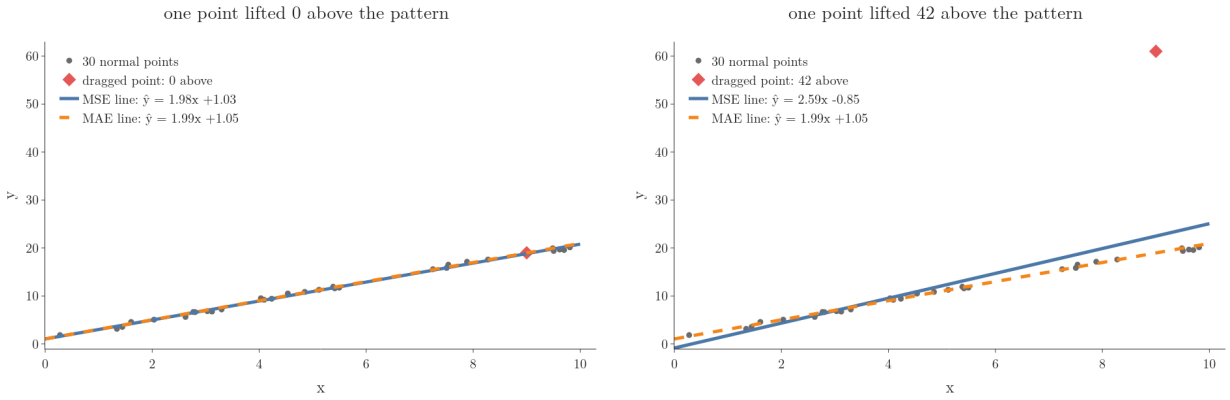


Figure 5: One point is lifted from the pattern to 42 above it; the line is refitted with MSE (blue) and MAE (orange) each time

Figure 5 uses the same 30 points as Figure 2, which follow $y = 2x + 1$, and drags one of them away. Watch the MSE line tilt towards the point, rising by 3.7 at $x = 9$, while the MAE line does not move at all.

Advantages:

- **Intuitive:** the average size of the error.
- **Same units as y:** an MAE of 2 means the predictions are off by 2 LPA on average.
- **Robust to outliers:** no square, so far observations do not dominate.

Disadvantage: the loss has a corner at an error of 0, where it has no derivative, and its slope jumps from -1 to $+1$ there. Gradient descent then needs a special rule at that point (a **subgradient**, see the derivatives of the perceptron loss), which makes the computation more awkward.

So: no outliers, use MSE; outliers in the data, use MAE. In Keras, `loss="mae"`, again with a linear output node.

7. Huber loss

> **Key point:** Huber loss is MSE for small errors and MAE for large ones. The threshold δ decides which errors count as **outliers** (G-1420).

7.1 When neither MSE nor MAE fits

> **Key point:** With many outliers, MSE follows them and MAE ignores them; Huber loss sits in between.

Suppose 25% of the points are **outliers** (G-1420): points far from the pattern the rest follow. So many outliers are no longer rare mistakes: they are part of the data. MSE gives them a huge pull; MAE treats them like any other point. **Huber loss** (G-905; Huber 1964) combines the two:

- if an observation's error is small ($|y - \hat{y}| \leq \delta$), it behaves like MSE;
- if the error is large, it behaves like MAE.

7.2 The formula

> **Key point:** $\frac{1}{2}e^2$ up to $|e| = \delta$, then a straight line $\delta(|e| - \frac{1}{2}\delta)$ that continues it smoothly.

1. **In words:** with error $e = y - \hat{y}$, use half the squared error while the error is within δ; beyond it, use a straight line that rises by δ per unit of error.
2. **Formula:**
$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \leq \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & |e| > \delta \end{cases}$$
3. **Example:** with $\delta = 1$, three errors:
$$e = 0.5: \quad \frac{1}{2} \times 0.5^2 = 0.125$$
$$e = 3: \quad 1 \times (3 - 0.5) = 2.5$$
$$e = 42: \quad 1 \times (42 - 0.5) = 41.5$$
The last one is the 50-lakh student: Huber charges 41.5, not 1764.

The two pieces meet at $|e| = \delta$ with the same value and the same slope, so the loss is smooth everywhere (Figure 4, green). The Huber slope is the error itself for small errors and $\pm\delta$ beyond, so no observation can push harder than δ.

δ is a hyperparameter: we try several values and keep the one that works best. In Keras: `loss=keras.losses.Huber(delta=1.0)`, which uses this same formula (Keras docs, Huber).

> **Extra:** The $\frac{1}{2}$ only makes the slope come out as e instead of $2e$. The factor changes the size of every update by the same factor, so it plays the same role as the learning rate.

7.3 The three losses on data with outliers

> **Key point:** On data with 25% **outliers** (G-1420), MSE lifts the line by about 2; MAE and Huber stay on the main points.

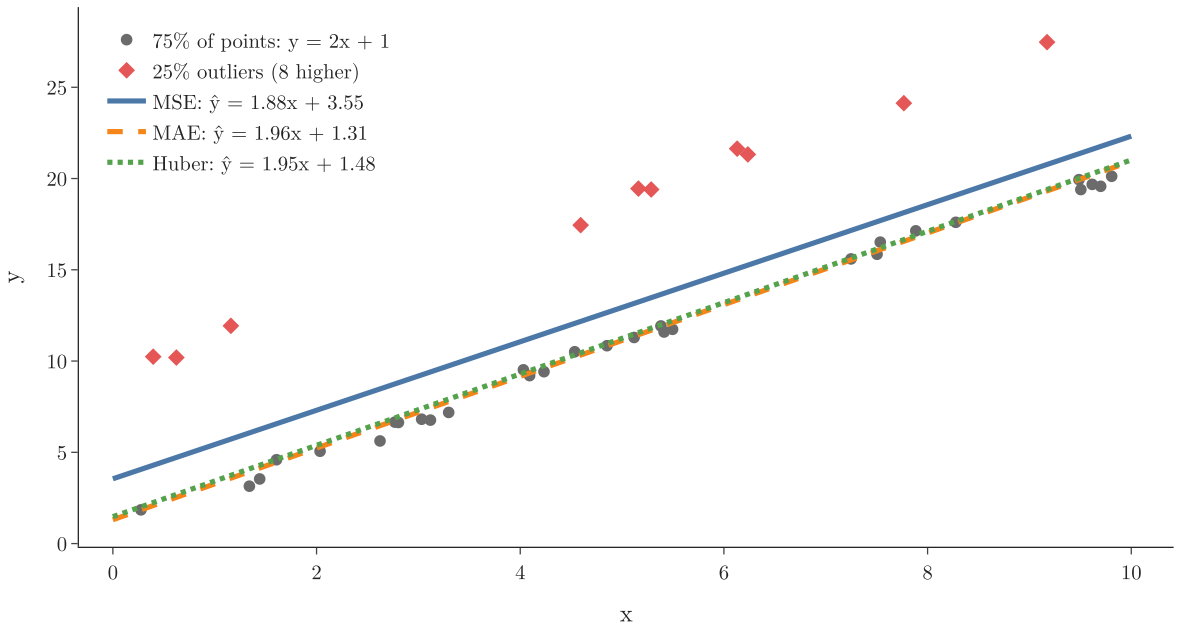


Figure 6: A line fitted with each loss to 40 points, 10 of them outliers

Figure 6 fits a straight line to 40 points: 30 follow $y = 2x + 1$, and 10 (25%) sit 8 units above it. Each loss gives a different line:

Loss	Fitted line
MSE	$\hat{y} = 1.88x + 3.55$
MAE	$\hat{y} = 1.96x + 1.31$
Huber, $\delta = 1$	$\hat{y} = 1.95x + 1.48$

MSE has moved the line up between the two groups, by about 2 (the outlier share 0.25 times the gap of about 8):

$$0.25 \times 8 = 2$$

MAE and Huber stay with the 75% of points that follow the pattern. With $\delta = 1$, Huber treats every outlier error of about 8 like MAE, so its line is close to MAE's. A larger δ moves it towards MSE's.

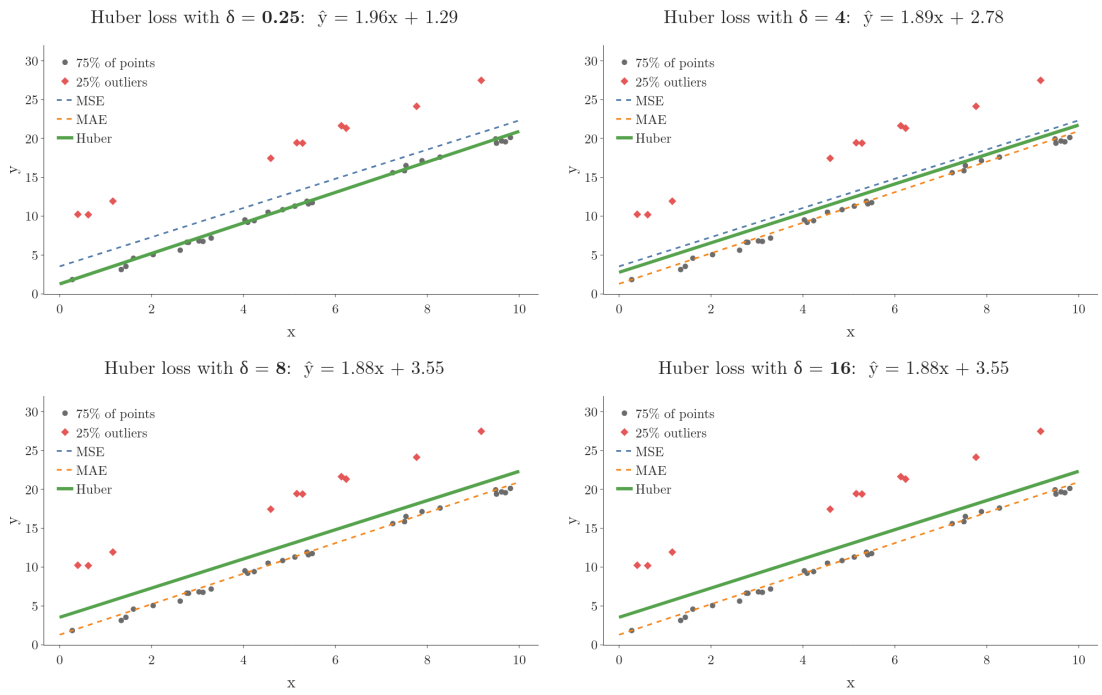


Figure 7: The Huber line (green) refitted on the 40 points of Figure 6 as δ grows from 0.25 to 16, with the MSE and MAE lines dashed

In Figure 7, watch the green line leave the orange MAE line and climb to the blue MSE line as δ grows: $\hat{y} = 1.96x + 1.29$ at $\delta = 0.25$, $1.89x + 2.78$ at $\delta = 4$, and $1.88x + 3.55$, exactly the MSE line, from $\delta = 6$ on. Once δ is larger than every error, no observation is treated as an outlier, and Huber loss is the squared error.

> **Extra:** Here Huber does not land halfway between MSE and MAE: it behaves like MSE only for errors below δ. Huber's real gain is that it is smooth (no corner at 0, unlike MAE) while still resisting outliers.

8. Binary cross-entropy

> **Key point:** For two classes. One sigmoid output node; loss $-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$.

When the output is a probability, the loss should be near 0 if the network gives the true class a high probability, and very large if it gives the true class a probability near 0. Minus the log of that probability behaves this way:

$$-\log 1 = 0$$

$$-\log 0.5 = 0.69$$

$$-\log 0.01 = 4.6$$

Binary cross-entropy (G-303), also called **log loss**, is this loss for two classes, and the loss of logistic regression. The formula and where it comes from (maximum likelihood) are in log loss:

$$L = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$J = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

We use it only when the output has two classes, such as placed (1) or not placed (0). In a network:

- the **output layer has one node with a sigmoid activation**, so $\hat{y}$ is a probability between 0 and 1;
- the hidden layers can use any activation.

Two students, with natural logs:

1. Student 1 is placed ($y = 1$) and the network gives $\hat{y} = 0.73$. Only the first term is on:
$$L = -\log 0.73 = 0.315$$
2. Student 2 is not placed ($y = 0$) and $\hat{y} = 0.25$. Only the second term is on:
$$L = -\log(1 - 0.25)$$
$$L = -\log 0.75 = 0.288$$

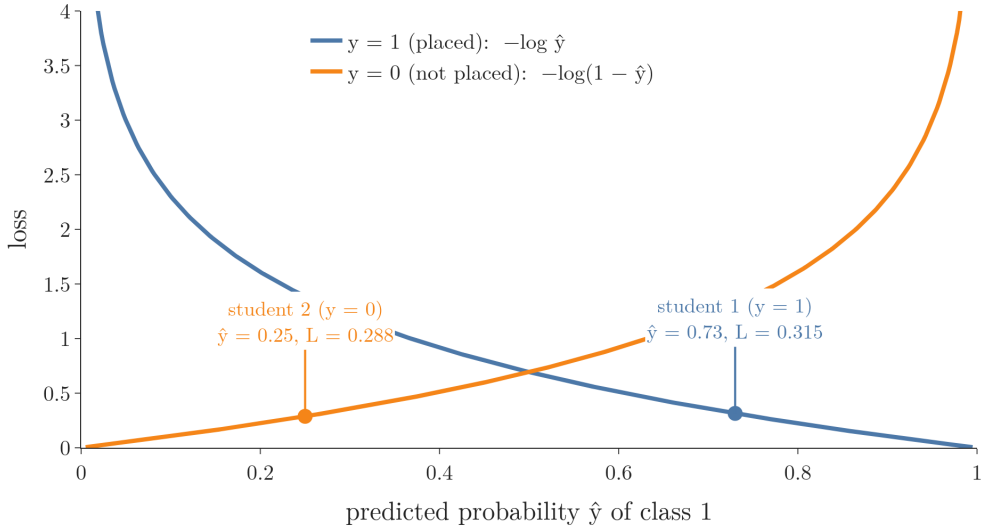


Figure 8: The loss for each label as the predicted probability changes, with the two students marked

Figure 8 draws the two halves of the formula. Watch each curve shoot up as the prediction moves towards the wrong label: a confident wrong answer costs far more than a hesitant one.

After each loss, gradient descent updates the weights, then the next student comes.

8.1 Why not the squared error?

> **Key point:** For a badly wrong probability, the slope of cross-entropy is huge and the slope of the squared error is small. The slope is the push on the weights, so cross-entropy corrects bad predictions much faster.

The squared error $(1-p)^2$ could also score a probability p given to the true class: it is 0 when $p = 1$ and largest when $p = 0$. The difference lies in the slope, which sets how hard the observation pushes the weights (section 5.2).

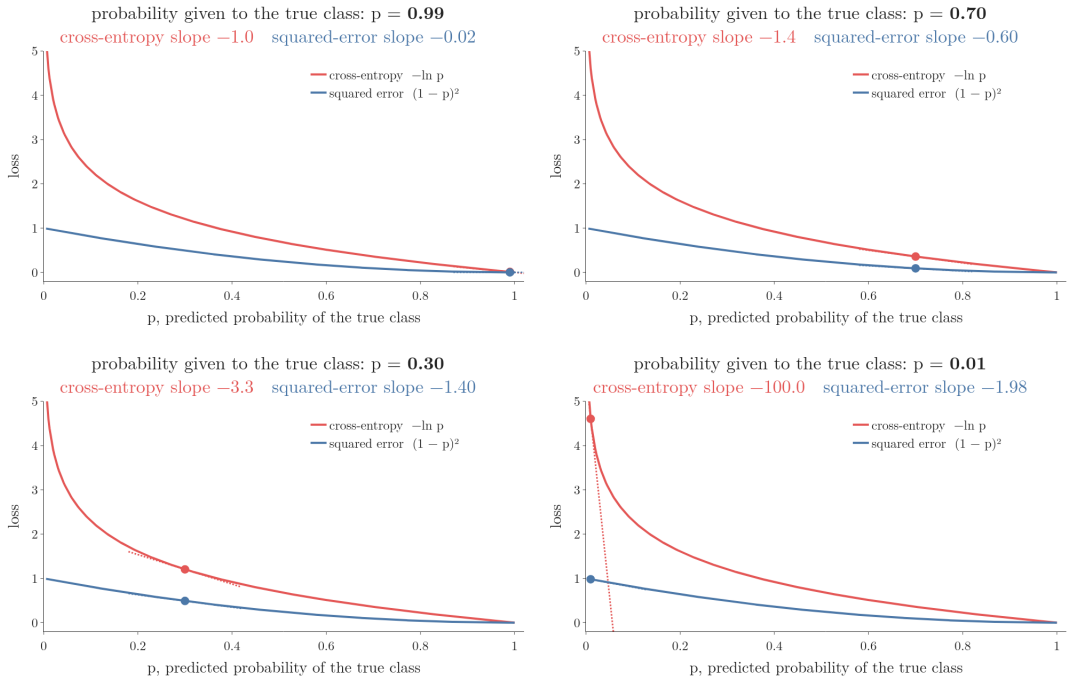


Figure 9: The probability p given to the true class slides from 0.99 to 0.01. Red: cross-entropy and its tangent. Blue: the squared error and its tangent. The title prints both slopes. Idea after StatQuest, "Neural Networks Part 6: Cross Entropy"

In Figure 9, watch the two dotted tangent lines as p falls:

1. **In words:** the slope of $-\log p$ is $-1/p$, and the slope of $(1-p)^2$ is $-2(1-p)$.
2. **Formula:**

$$\frac{d}{dp}(-\log p) = -\frac{1}{p}$$

$$\frac{d}{dp}(1-p)^2 = -2(1-p)$$

3. **Example:** for student 1, $p = 0.73$, the two slopes are:

$$-1/0.73 = -1.37$$

$$-2 \times 0.27 = -0.54$$

For a badly wrong prediction, $p = 0.01$:

$$-1/0.01 = -100$$

$$-2 \times 0.99 = -1.98$$

The squared error's slope can never be steeper than -2, however wrong the prediction. The cross-entropy's slope keeps growing as p approaches 0. So with cross-entropy, the worse the prediction, the larger the correction.

8.2 Advantages and disadvantages

- **Advantage:** it is differentiable, so gradient descent applies directly.
- **Disadvantage:** it is less intuitive than MSE or MAE: the number has no everyday unit.

Extra: The log here is the natural log, as in Keras.

Python: A binary classifier.

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(3, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="sgd", loss="binary_crossentropy")
```

9. Categorical cross-entropy

Key point: For more than two classes. One softmax output node per class; one-hot labels; only the log probability of the true class counts.

9.1 The setup

Key point: k classes means k output nodes with softmax, and one-hot encoded labels.

Suppose the output has three classes: placed *yes*, *no* or *maybe*. **Categorical cross-entropy** (G-349) is the loss of softmax regression (see the real approach: one loss for all classes). For one observation with k classes:

The symbol $\sum_{j=1}^{k}$ means "add the terms for $j = 1, 2, \ldots, k$"; here $k = 3$, so it adds three terms:

$$L = -\sum_{j=1}^{k} y_j \log \hat{y}_j$$

With $k = 3$ the sum written out is:

$$L = -y_1 \log \hat{y}_1 - y_2 \log \hat{y}_2 - y_3 \log \hat{y}_3$$

and the cost over n observations is:

$$J = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{k} y_{ij} \log \hat{y}_{ij}$$

The network changes in two ways:

- the **output layer has one node per class** (here 3: yes, no, maybe);
- they all use the **softmax** activation, so the three outputs lie between 0 and 1 and add up to 1 (see the softmax function):

$$\hat{y}_j = \frac{e^{z_j}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Why softmax, and not simply the largest raw output? Take three raw outputs of the output layer, 1.43, −0.4 and 0.23. They are hard to read: they are not between 0 and 1 and do not add up to 1. Two ways to tidy them are compared in Figure 10.

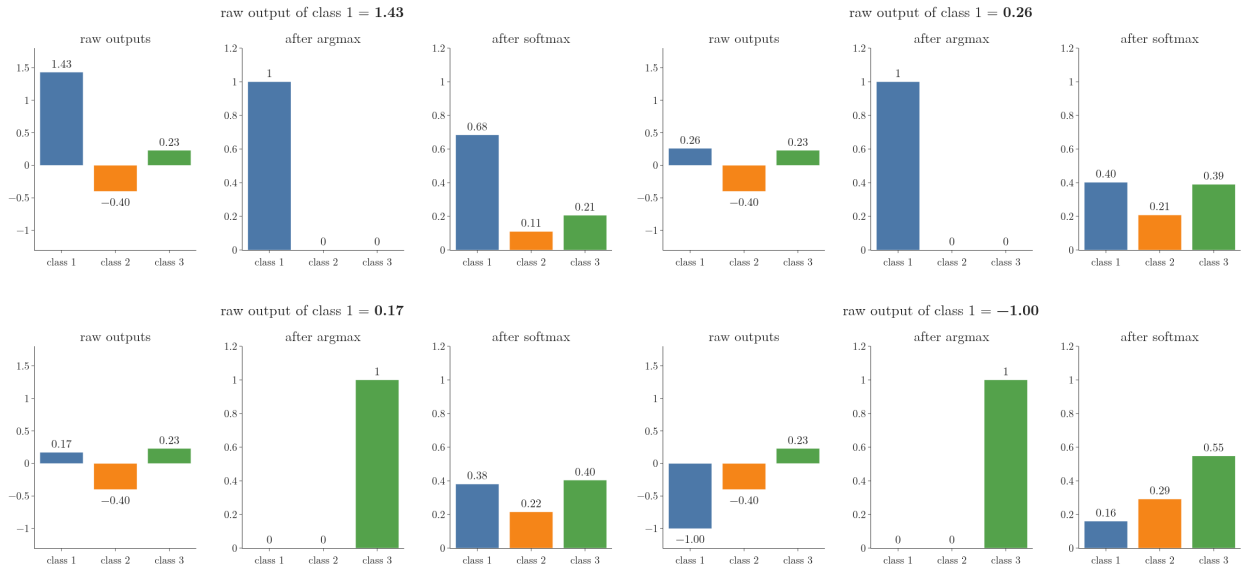


Figure 10: Three raw outputs (left), after argmax (middle) and after softmax (right), while the raw output of class 1 slides from 1.43 down to −1. Idea after StatQuest, "Neural Networks Part 5: ArgMax and SoftMax"

Softmax on those three raw outputs, one step per line:

$$e^{1.43} = 4.18$$
$$e^{-0.4} = 0.67$$
$$e^{0.23} = 1.26$$
$$4.18 + 0.67 + 1.26 = 6.11$$
$$\hat{y}_1 = \frac{4.18}{6.11} = 0.68$$
$$\hat{y}_2 = \frac{0.67}{6.11} = 0.11$$
$$\hat{y}_3 = \frac{1.26}{6.11} = 0.21$$

- **Argmax** sets the largest output to 1 and the rest to 0: here (1, 0, 0). The result is easy to read. But in Figure 10 the argmax bars stand still while the raw output of class 1 falls from 1.43 all the way to 0.26, and then jump at once when it drops below class 3's 0.23. A small change in a weight almost never changes the argmax output, so its slope is 0 and gradient descent gets no direction to move in.
- **Softmax** gives (0.68, 0.11, 0.21): the same order, every value between 0 and 1, and a sum of 1. In Figure 10 the softmax bars move at every step, so every small change in a weight changes the loss, and gradient descent has a slope to follow.

So training uses softmax, and argmax is used only afterwards, to read off the predicted class (see predicting with argmax).

The softmax outputs are called probabilities, but they are not a reliable measure of how sure the network should be: the handwritten-digit (MNIST) network gives random noise the label 3 with a softmax output of 0.997 (noise in, confident answer out).

The labels are one-hot encoded (one 0/1 column per class): yes = (1, 0, 0), no = (0, 1, 0), maybe = (0, 0, 1). A handwritten-digit network is the same idea with 10 classes (0 to 9) and 10 softmax output nodes.

9.2 Two students

> **Key point:** The one-hot label switches off every term except the true class.

1. Student 1 (CGPA 8, IQ 80) is a *yes*: $y = (1, 0, 0)$. The network outputs $\hat{y} = (0.2, 0.3, 0.5)$. Only y_1 is 1:
$$L = -1 \times \log 0.2 = 1.609$$
2. Student 2 (6, 60) is a *no*: $y = (0, 1, 0)$, and $\hat{y} = (0.3, 0.6, 0.1)$:
$$L = -1 \times \log 0.6 = 0.511$$

Student 1 gets a large loss because the network gave its true class only 0.2. After each loss, gradient descent updates the weights.

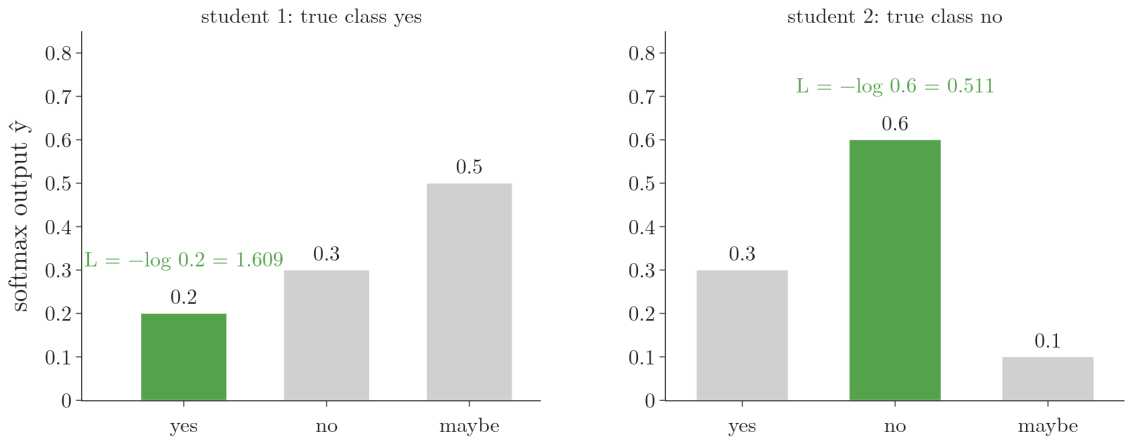


Figure 11: Softmax outputs of the two students; only the true class (green) enters the loss

In Figure 11, the grey bars play no part in the loss: only the green bar's height matters.

10. Sparse categorical cross-entropy

> **Key point:** The same loss as **categorical cross-entropy** (G-349), but the labels stay integers. No one-hot encoding is needed, and with many classes it saves memory and time.

Sparse categorical cross-entropy (G-1839) uses exactly the same network (one softmax node per class) and the same loss. Only the labels differ: *yes*, *no*, *maybe* become the integers 0, 1, 2 instead of one-hot vectors.

1. **In words:** read the true class from the integer label, pick that class's predicted probability, and take minus its log.
2. **Formula:** for an observation with label c,

$$L = -\log \hat{y}_c$$

3. **Example:** student 1 has label 0 and $\hat{y} = (0.2, 0.3, 0.5)$, so the loss picks the first probability:

$$L = -\log 0.2 = 1.609$$

Student 2 has label 1, so the loss picks the second probability:

$$L = -\log 0.6 = 0.511$$

These are exactly the numbers of Section 9.2.

The one-hot version multiplies the logs of all k classes by y_j, most of them by 0. The sparse version goes straight to one log (Figure 12).

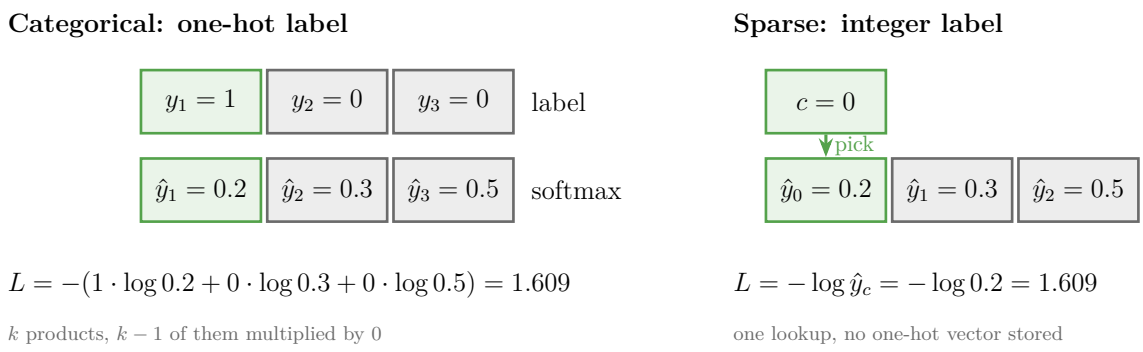


Figure 12: Student 1 under both label formats: three products, two of them zero, against one lookup

With 10 classes the difference is small; with thousands of classes (words in a vocabulary, for instance) it saves both the one-hot matrix and the wasted work.

Python: The two multi-class set-ups.

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(3, activation="relu"),
    keras.layers.Dense(3, activation="softmax"),  # one per class
])
# labels one-hot: [[1, 0, 0], [0, 1, 0], ...]
model.compile(optimizer="sgd", loss="categorical_crossentropy")
# labels as integers: [0, 1, ...]
model.compile(optimizer="sgd",
              loss="sparse_categorical_crossentropy")
```

Keras counts classes from 0. On the two students, both losses give the same average, 1.060. The Notebook checks every number of this Note with `keras.losses`.

11. Choosing a loss

Key point: Regression: MSE, or MAE with **outliers** (G-1420), or Huber with many. Two classes: binary cross-entropy. More classes: categorical cross-entropy, or its sparse version with many classes.

Loss functions are not limited to these. Other problems have their own losses (Figure 1):

- variational autoencoders: the KL divergence (Kingma and Welling 2014);
- GANs: a discriminator (min-max) loss (Goodfellow et al. 2014);
- object detection: the focal loss (Lin et al. 2017);
- embeddings: the triplet loss (Schroff et al. 2015);

- classification, as an alternative: the SVM's **hinge loss** (G-898).

Researchers design new losses for new problems, and Keras accepts our own loss functions too. Each loss has its strengths, and a poorly chosen one keeps the network from its best solution.

12. Summary

Problem	Output layer	Loss
Regression	1 node, linear	MSE
Regression with outliers	1 node, linear	MAE
Regression, many outliers	1 node, linear	Huber
Two classes	1 node, sigmoid	binary cross-entropy
k classes, one-hot labels	k nodes, softmax	categorical cross-entropy
k classes, integer labels	k nodes, softmax	sparse categorical cross-entropy

- The loss scores one observation; the cost averages the losses over a batch or the training set, so a small batch of losses is enough to estimate the cost and its gradient.
- MSE's slope grows with the error, so far observations drive the updates; MAE's slope is always ± 1, so **outliers** (G-1420) pull no harder than any other point.
- Huber loss is MSE inside $\pm\delta$ and MAE outside, and smooth everywhere, so it resists outliers like MAE without MAE's corner at 0.
- Cross-entropy's slope $-1/p$ grows without limit for a badly wrong probability; the squared error's slope stays within -2. The slope is the push on the weights, so cross-entropy corrects bad predictions much faster.
- Argmax has slope 0, so training uses softmax; argmax only reads off the final class.
- The loss decides the output layer: linear for MSE, MAE and Huber, sigmoid for **binary cross-entropy** (G-303), softmax for the categorical ones, because each loss needs its own kind of output: any number, one probability, or one probability per class.
- **Sparse categorical cross-entropy** (G-1839) is the same loss with integer labels, so the labels need no one-hot encoding.
- Keras names:
  - regression: `"mse"`, `"mae"`, `keras.losses.Huber(delta)`;
  - two classes: `"binary_crossentropy"`;
  - more classes: `"categorical_crossentropy"`;
  - more classes, integer labels: `"sparse_categorical_crossentropy"`.

So the problem type picks the loss, and the loss picks the output layer: the table above is the whole choice.

13. Sources

Built from

- CampusX, "Loss Functions in Deep Learning | Deep Learning | CampusX", YouTube, https://www.youtube.com/watch?v=gb5nm_3jBIo
- StatQuest with Josh Starmer, "Neural Networks Part 5: ArgMax and SoftMax", YouTube, https://www.youtube.com/watch?v=KpKog-L9veg (section 9.1: why softmax and not argmax)

- StatQuest with Josh Starmer, "Neural Networks Part 6: Cross Entropy", YouTube, https://www.youtube.com/watch?v=6ArSys5qHAU (section 8.1: cross-entropy against the squared error)

Other references

- Goodfellow, I., Bengio, Y. and Courville, A., *Deep Learning*, MIT Press, 2016, deeplearningbook.org, §5.9 "Stochastic Gradient Descent" (the cost is an average of per-example losses, so its gradient is an expectation that a minibatch estimates).
- Keras documentation, Losses (default reduction `"sum_over_batch_size"`: the average over the batch); `keras.losses.Huber`.
- Huber, P. J., "Robust Estimation of a Location Parameter", *Annals of Mathematical Statistics*, 1964.
- Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014.
- Goodfellow et al., "Generative Adversarial Nets", NeurIPS 2014.
- Lin, Goyal, Girshick, He and Dollár, "Focal Loss for Dense Object Detection", ICCV 2017.
- Schroff, Kalenichenko and Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering", CVPR 2015.

14. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Huber loss (G-905)	A regression loss that is half the squared error for errors up to δ and grows in a straight line beyond; it acts like MSE for small errors and like MAE for large ones, so outliers pull less.
δ (Huber) (G-8)	The error size at which Huber loss switches from squaring the error to taking its absolute value. It is a setting chosen before training (a hyperparameter).
KL divergence, focal loss, triplet loss (G-1014)	Losses for autoencoders, object detection and embeddings, taught with those networks.
L1 loss (G-1024)	Another name for the mean absolute error used as a training loss.
Squared loss (L2 loss) (G-1863)	The squared difference between target and prediction, $(y - \hat{y})^2$, used as the error a model lowers during training, so large errors cost much more than small ones; averaged over the rows it is the mean squared error.

Term	Meaning
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Mean squared error (MSE) (G-1201)	The average squared difference between actual and predicted values.
Mean absolute error (MAE) (G-1194)	The average absolute difference between actual and predicted values.
Binary cross entropy (log loss) (G-303)	The loss function of logistic regression for two classes: per row $-y \log a - (1-y)\log(1-a)$, averaged over the rows (the average cross entropy).
Categorical cross entropy (G-349)	The loss for classification with more than two classes and a softmax output: the average of minus the log of the probability given to the true class, $-\sum_j y_j \log \hat{y}_j$ per row with one-hot labels; it is small when the true class gets a high probability.
Loss function (error function) (G-706)	A formula that measures how wrong a model's predictions are; in deep learning, the error on one training row.
Cost function (G-492)	A measure of how far a model's predictions are from the true values, averaged over the data and read as a function of the model's parameters; training looks for the parameters that make it smallest. Often another name for the loss function.
Mini-batch gradient descent (G-1222)	Gradient descent that uses a small random group of rows for every update.
Outlier (G-1420)	A value far from the rest of the data.
Sparse categorical cross-entropy (G-1839)	The same loss as categorical cross-entropy, but it takes the labels as plain integers (such as the digits 0 to 9) instead of one-hot vectors, so the labels need no encoding.
Hinge loss (G-898)	The SVM's error for one point: 0 when the point is on the correct side and outside the margin, growing in a straight line the further it goes past the margin; formula $\max(0, 1 - y(w^T x + b))$.